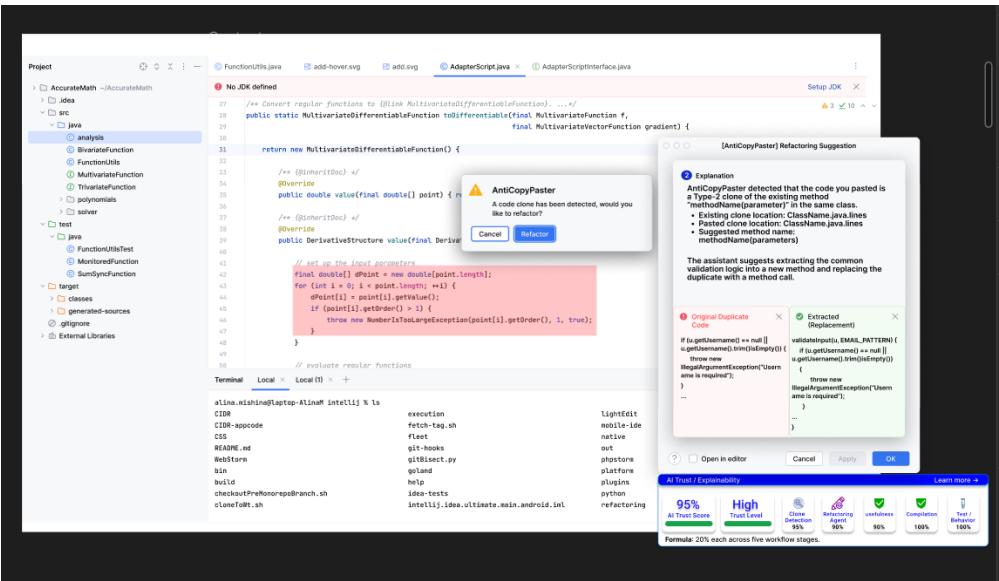


Log File Review and UI Design Process

AntiCopyPaster / Refactoring Detection Project

Purpose

This document organizes the log files collected while running the tool and explains how the information from those logs supports the current UI design. The goal is to show the process behind the design: what the tool reports internally, what problems appeared during testing, and what information should be visible to the developer in the new interface.



Final UI design reference: IntelliJ-style refactoring suggestion panel with explanation, diff preview, actions, and AI Trust / Explainability section.

Files Reviewed

Source	What it contributed
message.txt / Gemini 429 log	Showed quota and token-limit failures, retry delay information, and fallback behavior.
AprLifecycleListener DeepSeek log	Showed a more complete successful workflow: detection, refactoring, usefulness, compilation, testing, and final verification.
Running the tool + Log file.pdf	Showed a team test with clone detection, usefulness rejection, compilation success, EvoSuite test failure, and repeated quota errors.
Final UI screenshot	Provided the design direction that combines a popup, suggestion panel, diff view, and trust score area.

1. What the Logs Show About the Tool Process

The logs show that the tool works like a pipeline. Each stage creates information that can be shown in the UI so the developer understands what is happening instead of seeing a black-box AI result.

Stage	What happens in the logs	UI information to show
Paste detected	The log records the pasted snippet, file name, model, max attempts, and minimum clone count.	Show: Duplicate code detected after paste, file name, and selected code range.
Detection	Three detection panelists and a curator try to find clone ranges. If the LLM fails, PSI fallback can still find clones.	Show clone type, locations, line numbers, and whether detection came from LLM or fallback.
Refactoring	Panelists suggest Extract Method candidates. The curator selects the strongest option.	Show suggested helper method name and a short explanation of what will be replaced.
Usefulness check	The checker accepts or rejects the refactoring and gives reasons such as incomplete extraction or clone not replaced.	Show a clear pass/fail status and a human-friendly reason.
Compilation	The tool compiles the proposed source in isolation and reports whether build errors are new or pre-existing.	Show Compiles only when the proposed change is safe; separate new errors from existing project errors.
Testing	EvoSuite or tests run against the proposed code. Logs can show test failure, missing target class, or tool/environment failure.	Show Tests passed, Tests failed, or Tests unavailable with a short reason.
Final decision	The workflow either succeeds, fails after retries, or is verified but not applied if the user cancels.	Keep Apply disabled until safe; give Cancel / View Details / Open in editor actions.

Design takeaway

The UI should be organized around the same pipeline: detection, refactoring, usefulness, compilation, and testing. This is why the final design includes a suggestion panel, a diff preview, and an AI Trust / Explainability bar.

2. Important Information Found in the Logs

A. Quota and token-limit failures

Several runs failed because the Gemini API returned error 429. The status was `RESOURCE_EXHAUSTED`, and the logs included retry delays such as around 50 seconds. This means the UI should not only say failed; it should explain that the AI provider is unavailable because of quota or rate limits.

```
[LLM_REFACTOR] exception: Gemini error 429 status: RESOURCE_EXHAUSTED message: You exceeded your current quota...
RetryInfo: retryDelay around 50s
```

UI idea: add a clear error card such as: AI provider quota exceeded. Please retry later, switch model, or check API settings.

B. Fallback detection still worked when LLM calls failed

In one log, the LLM detection calls failed because of quota errors, but the PSI fallback still found clone ranges. This is important because the UI can tell the user that the result came from the fallback method instead of the AI panelists.

```
[DETECTION] no clones from LLM; trying PSI fallback (same-file) [DETECTION] PSI fallback candidates=2 [DETECTION] PSI
fallback accepted: ranges=2
```

UI idea: show a small badge like Detected by PSI fallback or AI unavailable - fallback used so the user understands why confidence may be lower.

C. Clone details are available and useful for the UI

The logs provide file names, clone ranges, class/method names, and detected occurrences. These details should be shown in the explanation section and in the expandable metadata area.

```
Clone candidate examples: RockPaperScissors#main [41-45, 48-52] Main#main [18-20, 22-24]
AprLifecycleListener#setSSLEngine / setSSLRandomSeed / setFIPSPMode [501-515, 526-540, 551-565]
```

UI idea: show Existing clone location and Pasted clone location with clickable line numbers that open the code.

3. What the Logs Teach Us About Trust and Validation

Usefulness check is very important

One run created a helper method, but the usefulness checker rejected it because the duplicate code was not fully replaced. This shows why the UI should not present every generated refactoring as safe. The UI should wait for validation and explain failures clearly.

Usefulness rejection example: Reason: EXTRACTION_WITHOUT_CLONE_REPLACEMENT_DETECTED Summary: A helper method was created, but the original duplicate blocks were not replaced. Feedback: Replace the duplicated loop blocks with calls to the new helper method.

UI idea: include a usefulness status line in the AI Trust section, for example: Usefulness failed - duplicate code still remains.

Compilation can pass even when tests fail

The logs show cases where compilation succeeded, but testing still failed. This means the UI should separate compilation and testing instead of combining them into one generic safe status.

[COMPILE] raw output: BUILD SUCCESS [COMPILE] ok (isolated) [TEST] status=tests_failed generatedTests=false Error: Could not locate test file for null

UI idea: show separate chips: Compilation passed and Tests failed/unavailable. This helps the developer understand exactly where the problem is.

Testing failures may be environment/tool failures, not code failures

One test failure came from EvoSuite and Java compatibility issues, such as Unsupported class file major version 69 and missing generated tests. This is different from the refactored code being behaviorally wrong.

EvoSuite issue example: Unsupported class file major version 69 generatedTests=false No statistics saved because EvoSuite failed to generate any test case

UI idea: add a third testing state: Tests unavailable or Testing tool failed so the user does not confuse an environment problem with a bad refactoring.

4. What a Successful Run Should Look Like in the UI

The DeepSeek log showed a more complete successful run. The tool detected three related clone occurrences, selected a refactoring candidate, passed usefulness, compiled, tested, and then reached success. This is the ideal flow the UI should communicate.

Log event	Important detail	UI element inspired by it
Detection success	Three clone ranges were detected in AprLifecycleListener.java.	Explanation section with clone count and file/line ranges.
Refactoring selected	Curator selected panelist P2 with confidence 0.95.	AI Trust section can show refactoring confidence and selected candidate.
Usefulness accepted	Usefulness curator accepted with confidence 1.0.	Validation badge: Useful Extract Method passed.
Compilation ok	Baseline errors existed, but proposed refactor compiled in isolation.	Compilation status should distinguish new errors from existing project errors.
Testing passed	The log ended with [TEST] passed and [WORKFLOW] SUCCESS.	Enable Apply button and show Safe to apply state.
User cancelled	The refactoring was verified but not applied because user cancelled.	Keep user in control with Cancel / Apply / Open in editor.

Important log excerpt

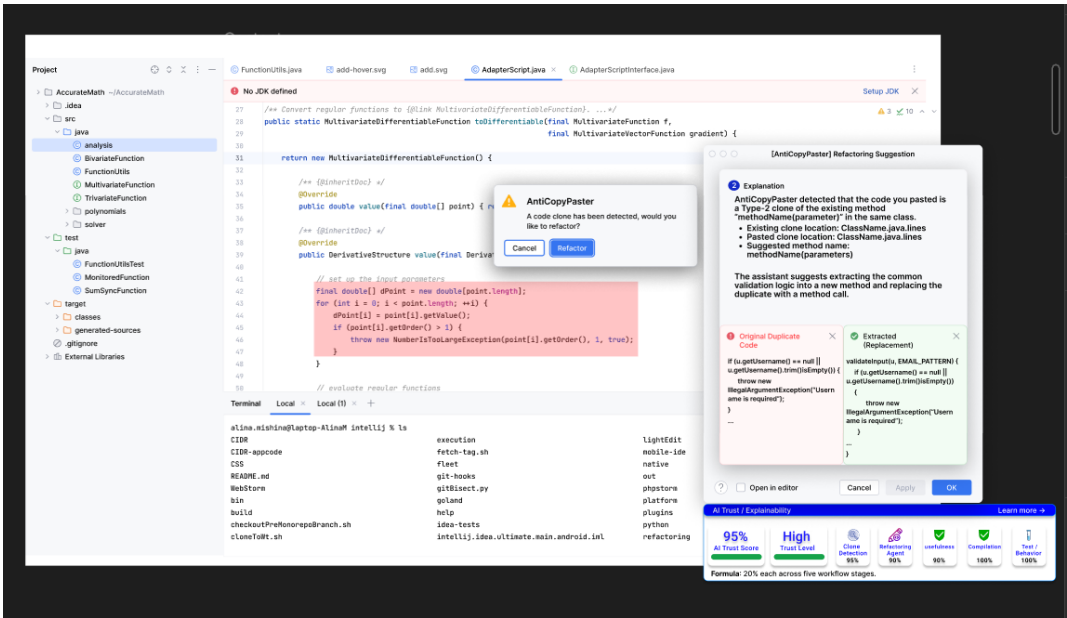
[LLM_REFACTOR] Curator selected P2, confidence=0.95 [USEFUL] LLM curator accepted proposal, confidence=1.0 [COMPILE] ok (isolated) [TEST] passed [WORKFLOW] SUCCESS

Design takeaway

A strong UI should make the success path obvious: the user should see detection, useful refactoring, compilation, and testing as separate steps that all passed before Apply becomes the main action.

5. How the Log Review Led to the Final UI Design

The final UI design uses the log information in a way that is easy for developers to understand inside IntelliJ. It does not show raw logs first; instead, it summarizes the important parts and leaves technical details expandable.



Log information	Final UI feature
File name, clone ranges, pasted snippet	Explanation area with clone location and pasted location.
Suggested helper method and replacement code	Diff preview with original duplicate code and extracted method replacement.
Usefulness, compilation, and testing results	AI Trust / Explainability bar with separate validation chips.
Retry attempts and quota failures	Loading state, refactoring attempts counter, and failure-state messages.
API key / provider errors	Settings validation and provider-specific error popups.
Raw log details	Expandable technical details instead of cluttering the main panel.

6. UI Ideas to Add Based on the Logs

UI idea	Why it matters
Stage progress indicator	Show the current step: Detection -> Refactoring -> Usefulness -> Compilation -> Testing. This matches the log structure and helps users know where the tool is.
Attempt counter	The logs use maxAttempts=3 and show attempt 1/3, 2/3, and 3/3. The loading UI should show how many attempts have been made.
Provider/model status	Show the AI provider and model being used, plus a warning if it is unavailable because of quota or token limits.
Clear failure-state messages	Differentiate between bad refactoring, API quota, test generation failure, and existing project compilation errors.
Confidence / trust breakdown	Base the trust score on workflow evidence: clone detection, refactoring quality, usefulness result, compilation result, and testing behavior.
Expandable technical details	Keep the main panel simple, but allow advanced users to open log details, panelist decisions, clone ranges, and retry history.
Disable Apply until safe	Apply should stay disabled if usefulness, compilation, or testing is unresolved or failed.
Settings/API key helper	Show API key validation hints and allow AQ. keys or remove strict validation if that is part of the code update.

Recommended AI Trust Score components

Based on the latest direction from the team, the UI should not rely only on model confidence. A better trust score should be based on evidence from the workflow itself: clone detection, refactoring quality, refactoring usefulness, compilation, and testing/behavior. If any stage is unavailable because of API quota or tool failure, the UI should show that the trust score is incomplete instead of pretending the result is fully safe.

7. Summary for the Team

The log files helped the team move from a general prototype to a more evidence-based UI design. They showed exactly what the tool knows internally and what users need to see before trusting a refactoring suggestion.

Main conclusions

- The UI should show the pipeline clearly instead of hiding the process.
- Error messages need to explain the cause: quota, API key, usefulness failure, compilation issue, or test tool failure.
- Validation should be separated into usefulness, compilation, and testing because each stage can succeed or fail independently.
- The AI Trust / Explainability section should be based on real workflow evidence from the logs.
- Developers should stay in control: they should be able to view the diff, open the code, cancel, or apply only after the suggestion is validated.

Final design direction

The final UI should feel like a normal IntelliJ refactoring assistant, but with extra explainability for AI-generated suggestions: a simple confirmation popup, a detailed suggestion panel, a diff preview, validation chips, trust score details, and clear failure states.